PostgreSQL Storage Layer

Project Document: Day 4 — PostgreSQL Storage Layer & Schema Design

1. Overview and Objectives

This document outlines the architecture and implementation of the relational database layer for the Global Gateway RS data pipeline.

Building upon the **Phase 2 Data Cleaning & Feature Engineering** pipeline completed previously—which introduced advanced NLP features such as VADER sentiment scoring, spaCy-based Named Entity Recognition (NER), and MiniLM-L6-v2 semantic clustering—Day 4 focuses on persisting these complex data structures efficiently. The storage layer utilizes PostgreSQL for production, leveraging native `JSONB` types for arrays and `TSVECTOR` for high-performance full-text search, alongside an SQLAlchemy Object-Relational Mapping (ORM) and a robust repository pattern.

2. File Structure & Component Breakdown

The storage layer is modularized into three primary components:

- **`models.py`**: Contains the SQLAlchemy Declarative Base and the `Article` schema definition. It maps Python data types to PostgreSQL-specific dialects.
- **`repository.py`**: Implements the `ArticleRepository` class. This abstracts the database connection (`SessionLocal`) and provides standard CRUD operations (`insert_article` , `get_article_by_hash` , `update_article` , `delete_article`) with built-in transaction rollback and integrity error handling.
- **`alembic/` (Directory)**: Houses the migration environment, `alembic.ini` , and version-controlled migration scripts for schema evolution.

3. Database Schema Specification (`articles` table)

The schema is explicitly designed to handle both the raw scraped data from V1 and the enriched NLP features from Phase 2.

3.1. Core Metadata

- **`id`** *(Integer, Primary Key)*: Auto-incrementing internal identifier.

- `url_hash` *(String 32, Unique, Indexed)*: MD5 hash of the article URL to enforce strict deduplication and act as the primary lookup key.
- `url` *(String)*: The original source URL.
- `title` *(String)*: Scraped article title.
- `description` *(Text)*: Scraped or LLM-extracted summary/description.
- `author` *(String)*: Article author, defaulting to "Unknown".
- `published_at` *(DateTime, Indexed)*: Standardized publication timestamp.
- `top_image` *(String)*: URL to the primary article image.

3.2. Phase 2 Enriched Features (NLP & Content Metrics)

- `is_clean` *(Boolean)*: Quality flag generated during Stage 2 text filtering.
- `word_count` *(Integer)*: Calculated length of the description text.
- `reading_time_mins` *(Integer)*: Estimated reading time based on a 200 WPM average.
- `keyword_density` *(Float)*: Ratio of tags found within the description.
- `tags` *(JSONB)*: Array of standardized tags extracted by the Groq LLaMA 3.1 model.
- `sentiment_score` *(Float)*: VADER compound sentiment score (-1.0 to 1.0).
- `sentiment_label_vader` *(String)*: Categorical label (positive, neutral, negative).

3.3. Named Entity Recognition (NER)

- `entities_persons` *(JSONB)*: Array of people identified via spaCy.
- `entities_organizations` *(JSONB)*: Array of organizations identified via spaCy.
- `entities_locations` *(JSONB)*: Array of locations identified via spaCy.

3.4. Source Metrics & Semantic Clustering

- `category_id` / `source_id` *(Integer, Indexed)*: Foreign keys for relational mapping to potential future `Categories` and `Sources` tables.
- `source_region` *(String)*: Geographical mapping of the source (e.g., UAE, Global).
- `source_tier` *(Integer)*: Assigned editorial credibility weight (1, 2, or 3).
- `sub_category` *(String)*: Granular rule-based category matching.
- `story_id` *(Integer)*: Semantic cluster ID assigned via cosine similarity matching on 384-dimensional dense embeddings (preventing redundant recommendations).

3.5. Full-Text Search Configuration

- `search_vector` *(TSVECTOR)*: A specialized PostgreSQL vector type column.
- **Trigger Implementation**: An Alembic raw SQL execution (`article_search_vector_update`) automatically populates this vector on `INSERT` or `UPDATE` by concatenating the weighted `title` (Weight A) and `description` (Weight B).
- **Indexing**: Utilizes a Generalized Inverted Index (GIN) on the `search_vector` for highly optimized text queries.

4. Local Development Fallback (SQLite)

To support offline development and local analytical workflows (matching the `news_cleaned.db` outputs from Phase 2), an isolated SQLite implementation is maintained.

Variations from Production:

- Because SQLite lacks native `JSONB` , Python lists (tags, entities) must be explicitly serialized into JSON strings via `json.dumps()` before insertion.
- The `TSVECTOR` and GIN index functionalities are omitted in the SQLite environment, relying instead on standard text queries.
- Execution is handled via a lightweight script utilizing the native `sqlite3` library.